

Collections framework

COMPLETE LEARNING GUIDE

What You'll Learn

The Java Collections Framework is arguably the single most interview-tested topic in the entire Java curriculum, because knowing which collection to reach for — and why — signals real engineering judgment rather than memorized syntax. This guide covers the core hierarchy (List, Set, Queue, Map), the concrete implementations you'll use constantly (ArrayList, LinkedList, HashSet, TreeSet, LinkedHashSet, HashMap, TreeMap, LinkedHashMap) with their actual internal data structures and Big-O complexity for common operations, how HashMap resolves collisions and rehashes as it grows, the ConcurrentModificationException trap when mutating a collection while iterating it, and the Comparable/Comparator distinction for custom ordering. Every one of these details has appeared verbatim in real placement interview questions — "what's the time complexity of HashMap.get()" and "why does ArrayList insertion at the front cost $O(n)$ " are asked constantly.

1. The Core Hierarchy: Collection vs. Map

The Collections Framework splits into two parallel hierarchies that do not share a common root: the Collection interface (extended by List, Set, and Queue) represents a group of individual elements, while Map represents key-value pairs and is deliberately not a Collection, because its fundamental operation (lookup by key) differs from iterating a flat sequence of elements. List preserves insertion order and allows duplicates and index-based access; Set enforces uniqueness (no duplicates, by equals()/hashCode()); Queue models FIFO (or priority-based) processing order. Every concrete class you'll use — ArrayList, HashSet, HashMap — implements one of these interfaces, and interview questions are almost always phrased in terms of the interface ("when would you use a Set over a List") rather than the concrete class.

2. List Implementations: ArrayList vs. LinkedList

ArrayList is backed by a dynamically resized array: random access by index is $O(1)$ since it's direct array indexing, but inserting or removing at the front or middle is $O(n)$ because every subsequent element must shift. LinkedList is backed by a doubly-linked list of nodes: insertion/removal at a known position (via an iterator) is $O(1)$ since it's just pointer relinking, but random access by index is $O(n)$ because reaching index i requires walking i nodes from one end. In practice, ArrayList is the default choice for almost all use cases because modern CPU cache locality makes its array-based access faster in real-world benchmarks even for many "insert in the middle" workloads — LinkedList is rarely the right answer despite being the "textbook" choice for insertion-heavy code.

- ArrayList: $O(1)$ get(index) and add-at-end (amortized), $O(n)$ insert/remove at front or middle.
- LinkedList: $O(1)$ insert/remove given a node reference/iterator, $O(n)$ get(index) (must traverse).
- ArrayList's dynamic resize (grow() when full) copies all elements to a new, larger backing array — an $O(n)$ operation, but amortized $O(1)$ per add() over many calls.
- Vector is a legacy synchronized alternative to ArrayList — effectively obsolete since Java 5's concurrent collections are faster.

3. Set Implementations: HashSet, LinkedHashSet, TreeSet

HashSet stores elements in a hash table, giving $O(1)$ average-case add/remove/contains but no guaranteed iteration order at all — two runs of the same program can even iterate in different orders. LinkedHashSet adds a doubly-linked list threading through the hash table entries to preserve insertion order at a modest memory and performance cost over HashSet. TreeSet is backed by a red-black tree (a self-balancing binary search tree), keeping elements in sorted order at all times, which costs $O(\log n)$ for add/remove/contains instead of HashSet's $O(1)$ — you pay for ordering with logarithmic operations.

- HashSet: $O(1)$ average add/remove/contains, no ordering guarantee — the default choice for pure uniqueness checks.
- LinkedHashSet: $O(1)$ like HashSet, but iterates in insertion order.
- TreeSet: $O(\log n)$ operations, always iterates in sorted order (natural ordering or a supplied Comparator).
- All Set implementations reject duplicates based on equals()/hashCode() (HashSet/LinkedHashSet) or compareTo() (TreeSet) — note TreeSet can treat two "equal by compareTo" but "unequal by equals" objects as duplicates.

4. HashMap Internals: Buckets, Collisions, and Treeification

A HashMap stores entries in an array of buckets; a key's hashCode() is run through an internal hash-spreading function and reduced modulo the array length to pick a bucket index. Multiple keys landing in the same bucket (a collision) are historically stored as a linked list of entries within that bucket, giving $O(1)$ average-case get()/put() but $O(n)$ worst-case if many keys collide — since Java 8, a bucket that accumulates 8 or more colliding entries (with a table of at least 64 buckets) converts that bucket's list into a small red-black tree, bounding the worst case to $O(\log n)$ instead of $O(n)$. The map automatically resizes (doubling the bucket array and rehashing every entry) once the number of entries exceeds capacity $\times$ loadFactor (default 0.75), an $O(n)$ operation that happens rarely enough to keep put() amortized $O(1)$.

JAVA

```
Map<String, Integer> wordCounts = new HashMap<>();
for (String word : words) {
    wordCounts.merge(word, 1, Integer::sum);    // increments count, or inserts 1
}
// get/put/containsKey are all O(1) average case
```

5. Map Ordering Variants: TreeMap and LinkedHashMap

TreeMap is HashMap's sorted counterpart — backed by a red-black tree, keeping keys in sorted order (natural or via a Comparator) at the cost of $O(\log n)$ instead of $O(1)$ for get/put, and additionally offering navigation methods (firstKey(), floorKey(), ceilingEntry()) that a plain HashMap cannot provide. LinkedHashMap preserves insertion order (or, configured with accessOrder=true, least-recently-used order) with the same $O(1)$ average performance as HashMap, and is the standard building block for implementing an LRU cache, since removeEldestEntry() can be overridden to automatically evict the oldest entry once a size threshold is crossed.

6. Iteration, fail-fast Behavior, and ConcurrentModificationException

Java's standard collection iterators are fail-fast: they track a modCount on the underlying collection, and if the collection is structurally modified (elements added or removed, not merely have their values changed) by any means other than the iterator's own remove() method while iteration is in progress, the very next call to next() throws ConcurrentModificationException. This is a deliberate safety mechanism, not a bug — removing an element directly from a List inside a for-each loop over that same list is one of the most common runtime exceptions new Java developers encounter, and the fix is to use the Iterator's own remove() method, or collect items to remove into a separate list and remove them afterward, or use removeIf().

JAVA

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5));

// WRONG -- throws ConcurrentModificationException
// for (int n : nums) { if (n % 2 == 0) nums.remove(Integer.valueOf(n)); }

// CORRECT -- Iterator's own remove()
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}

// CORRECT and idiomatic -- removeIf()
nums.removeIf(n -> n % 2 == 0);
```

7. Comparable vs. Comparator for Custom Ordering

Comparable is implemented by the class itself (public class Employee implements Comparable<Employee>), defining that type's single "natural ordering" via compareTo(); it's used automatically by Collections.sort(), TreeSet, and TreeMap when no other ordering is specified. Comparator is a separate object that defines an ordering externally to the class (useful when you don't own the class's source, or need multiple different orderings), passed explicitly to sort() or a TreeSet/TreeMap constructor — modern Java's Comparator.comparing() and thenComparing() let you build multi-field comparators fluently without writing a full compareTo() method by hand.

JAVA

```
// Comparable: one natural ordering, defined inside the class
public class Employee implements Comparable<Employee> {
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary);
    }
}

// Comparator: external, supports multiple orderings, composable
employees.sort(Comparator.comparing(Employee::getLastName)
    .thenComparing(Employee::getFirstName));
```

Common Pitfalls

- Removing an element from a List/Set directly inside a for-each loop over it, triggering a ConcurrentModificationException — use Iterator.remove() or removeIf() instead.
- Choosing LinkedList "because insertion should be O(1)" without accounting for its O(n) random access, when ArrayList is faster in most real workloads.
- Assuming HashMap/HashSet iteration order is meaningful or stable — it is unspecified and can change between JVM versions or even between runs.
- Using a mutable object as a HashMap key or HashSet element, then mutating it after insertion — this can make the entry permanently unfindable since its hash bucket no longer matches its current state.
- Overriding equals() without hashCode() (or vice versa) on a class used as a HashMap key, silently breaking lookups.
- Expecting TreeSet/TreeMap to use equals() for duplicate detection — they actually use compareTo()/Comparator, which can disagree with equals() if not implemented consistently.
- Forgetting that HashMap's default load factor (0.75) triggers a full O(n) rehash on resize — pre-sizing a HashMap's initial capacity when the final size is roughly known avoids repeated rehashing.

Key Takeaways

- ArrayList: $O(1)$ indexed access, $O(n)$ insert/remove in the middle; LinkedList: the reverse — $O(1)$ insert/remove at a known node, $O(n)$ indexed access.
- HashMap/HashSet: $O(1)$ average case, no ordering guarantee; TreeMap/TreeSet: $O(\log n)$, always sorted; LinkedHashMap/LinkedHashSet: $O(1)$, insertion order preserved.
- HashMap resolves collisions with a linked list per bucket (treeified into a red-black tree past 8 collisions in one bucket since Java 8) and resizes at 75% load factor.
- Never structurally modify a collection while iterating it with a for-each loop — use `Iterator.remove()` or `removeIf()` to avoid `ConcurrentModificationException`.
- Comparable defines one natural ordering inside the class; Comparator defines external, composable, and multiple orderings.
- LinkedHashMap with `accessOrder=true` and an overridden `removeEldestEntry()` is the standard way to implement an LRU cache in plain Java.

Learn More

Oracle Java Tutorials — The Collections Framework

<https://docs.oracle.com/javase/tutorial/collections/index.html>

Oracle Java SE Docs — HashMap

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html>

GeeksforGeeks — Internal Working of HashMap in Java

<https://www.geeksforgeeks.org/java/internal-working-of-hashmap-java/>